The Urban Gazette

VOL. 1, NO. 024

2026-03-30

FEATURES

Debugging custom ApiGateway authorizers

Luciano Mammino · Luciano Mammino (loige)

Last week, while working on a custom REST API Gateway Lambda authorizer , I spent some time trying to debug a mysterious 500 error with a {“message”:null} body. In this article, I will share why this was happening and how to debug this kind of error when building custom API Gateway authorizers.

The use case

Before diving into the problem, let me give you a bit of context about the use case I was working on. As I said, I was working on an open-source project implementing a custom authorizer for AWS API Gateway. The project is simply called oidc-authorizer and it’s already available on GitHub and the Serverless Application Repository (SAR) . This authorizer is implemented as a Lambda function and it allows you to authenticate requests following to OIDC (OpenID Connect) protocol.

To better understand what that means, let’s have a look at this lovely diagram (that I am proud to have drawn by myself):

In this diagram, we see a user that sends an authenticated request to API Gateway. API Gateway is configured to use a lambda as a custom authorizer. The lambda talks with a given OIDC provider to get the public key to validate the user token and responds to API Gateway to Allow or Deny the request.

This type of authentication is based on the idea that a token (in this case, a JWT) can be trusted only if it’s signed by an authority we trust. That’s why we need to talk with the OIDC provider to get the public key to validate the signature of the token.

I have been speaking and writing at length about JWT before, so I am just going to say that if you want to deep dive into the fascinating topic of OIDC you can check out the official specification of the OIDC standard . If you have the guts to read protocol specifications, it’s a very interesting read, I promise! 😇

The problem

Ok, all pretty cool, but what was the problem?

The problem is that I am a terrible programmer and when I started testing my first version of the authorizer my code was quite buggy and it didn’t work in the way that REST API Gateway Authorizers are supposed to work.

I take responsibility for that, but that’s only part of the problem. The other side of the coin is that AWS wasn’t really giving me a useful error message: the only thing I could see when making a request was a 500 error with a response body containing only the JSON payload {“message”:null} .

The full response:

HTTP/2 500

content-type: application/json

content-length: 16

date: Sat, 04 Nov 2023 12:13:50 GMT

x-amzn-requestid: [...]

x-amzn-errortype: AuthorizerConfigurationException

x-amz-apigw-id: w218poymg7

x-cache: Error from cloudfront

via: 1.1 [...]cloudfront.net (CloudFront)

x-amz-cf-pop: [...]

x-amz-cf-id: [...]

{“message”:null}

Shrugh... What do you do in these cases? 🤷

You try to look for some logs in CloudWatch, right? Well, I did that and I found nothing. API Gateway doesn’t have a log group by default and I was expecting to see some logs from my Lambda function, but there was nothing useful there!

The solution

The solution is to enable extensive logging for API Gateway.

So, how do we do that?

We can do that using Infrastructure as code using CloudFormation or SAM.

These are the logical steps we need to follow:

Create a Role that allows API Gateway (at the service level) to write logs to CloudWatch

Use the AWS:: ApiGateway:: Account resource to assign the role we just created to API Gateway.

Updated the specific instance of a REST API Gateway stage to enable logging and tracing on it.

Enabling API Gateway logging to CloudWatch with CloudFormation

OK, let’s start with the code for the first 2 steps using the following CloudFormation template:

apigw-logging.yaml

```
AWSTemplateFormatVersion : '2010-09-09'
```


(Edited for length)

To avoid overwriting other roles, we should only have one AWS:: ApiGateway:: Account resource per region per

JavaScript, low-level or AI?

Luciano Mammino · Luciano Mammino (loige)

The software industry is going to be fun to see in the coming 5-10 years.

I see an interesting tension happening right now...

The generative AI side

On one side we have generative AI capturing almost every bit of the software development lifecycle and effectively becoming a new high-level abstraction, possibly the highest level we have ever seen in our industry.

There seems to be no stopping this phenomenon... just looking at the latest announcements from GitHub Universe, it's clear that we will have to adopt AI in one way or another and this is not necessarily a bad thing if it truly makes us all more productive and focused on generating business value.

If you don't know what I am talking about you should check out this video:

And if that's not enough you should watch the full GitHub Universe 2023 keynote .

GitHub Universe 2023, Day 1. Thomas Dohmke on stage with a slide in the background saying "One more thing", mimicking the Steve Job's launch of the iPad

The part that impressed me the most is that now Copilot can also help you lay out the structure of a project, breaking down requirements into potential tasks. Once you are happy with the result, it can start to work on the individual tasks and submit PRs. This is called Copilot workspace , if you want to have a look.

It seems too good to be true and it's probably going to be far from perfect for a while, but there's great potential for efficiency here, and I am sure GitHub (and other competitors) will keep investing in this kind of products and maybe in a few years, we'll be mostly reviewing and merge AI-generated PRs for the most common use cases.

If you think that ChatGPT was launched slightly less than 1 year ago, what will we be seeing in 5 or 10 years from now?

The low-level side

On the other side, we have a wave of new low-level languages such as Go , Rust , Zig (and Carbon , and Nim , and Odin , and VLang , and Pony , and Hare , and Crystal , and Julia , and Mojo , and.. I could keep going here... ♀).

OK, I really wanted to put the lovely Hare language mascot here. Hare is a systems programming language designed to be simple, stable, and robust. Hare uses a static type system, manual memory management, and a minimal runtime. It is well-suited to writing operating systems, system tools, compilers, networking software, and other low-level, high-performance tasks.

All these languages take slightly different trade-offs, but, at the end of the day, they are built on the premise that we need to go lower level and have more fine-grained control over how we use memory, CPU, GPU and all the other resources available on the hardware. This is perceived as an important step to achieve better performance, lower production costs, and reach the dream of "greener" computing.

If you are curious to know why we should be caring about green computing , let's just have a quick look at this report: Data Centres Metered Electricity Consumption 2022 (Republic of Ireland) .

The report findings state that in 2022, in Ireland alone, data centers' energy consumption increased by 31%. This increase amounts to an additional 4,016 Gigawatt/hours. To put that in perspective we are talking about the equivalent of an additional 401.600.000.000 (402 billion!) LED light bulbs being lit every single hour . If you divide that by the population of the Republic of Ireland this is like every individual is powering 80.000 additional LED light bulbs in their home, all day and night! And this is just the increase from 2021 to 2022... How friggin' crazy is that?!

Ok, now one could argue that we had low-level languages pretty much since the software industry was invented. So why isn't that the default and why do we bother wasting energy with higher-level programming languages?

That's actually quite simple: because coding in low-level programming languages such as C and C++ is hard! Like really really hard! And it's also time-consuming and therefore expensive for companies! And I am not even going to mention the risk of security issues that come with these languages.

So why should this new wave of low-level programming language change things?

> *And I would go as far as saying that these use cases exist in the industry today and there's a staggering lack of talent in these areas.*
>
> Luciano Mammino

Well, my answer is that they are trying to make low-level programming more accessible and safe. They are trying to create paradigms that could be friendly enough to be used for general computing problems (not just low-level), which could potentially bring the benefits of performance and efficiency even in areas where historically we have been using higher-level languages and made the hard tradeoff of fast development times vs sub-optimal performance.

Take for example Rust. It was historically born to solve some of the hard problems that Mozilla had to face while

Invite-only microsites with Next.js and AirTable

Luciano Mammino · Luciano Mammino (loige)

I recently needed to create an invite-only Next.js-powered microsite (for my wedding) and in this article I'm going to show you how I went about creating invite codes and how I implemented code validation in the app, using Next.js API endpoints and AirTable as a lightweight backend.

The use case and the tech stack

24 June 2022 was the best day of my life.

Seriously, I got married with the human being I love the most and everything was just perfect.

But we are not here to talk about my wedding, right?

Well, we kinda are, but in a nerdy kind of way that only software engineers can (hopefully) appreciate. OK, don't tell my wife I said that...

Specifically, we will discuss how to implement an invite-only microsite to share information with all the people invited to a private event.

Just to put things in context, let's wear the PM hat for a second, and let's discuss what kind of requirements did I impose on myself while building my wedding website:

I wanted to be able to iterate quickly, mostly focusing on content and design, not much on infrastructure.

I wanted to build a simple solution that would be easy to host, maintain and update.

I wanted to have a very lightweight backend, possibly something managed.

Even better if I could give access to the data storage to someone else (my wife) so that we could collaborate on managing the data about the guests.

If I could host everything very cheaply (if not even freely), that would be great.

For all the reasons above I settled for adopting the following pieces of tech:

Next.js for building a statically rendered frontend and the backend APIs

A private GitHub repository for managing the code of the Next.js app

Vercel for hosting the project

AirTable to store and manage all the data (invite codes, guests, rsvp, etc.)

Keep in mind that Next.js is free and open source, GitHub can be used for free (even if you want to keep your repo private), and Vercel and AirTable have both generous free plans that are more than enough for this kind of use case.

With the stack above I ended up spending 0 money and I was able to complete the project in a couple of weekends!

Making a Next.js site private

On top of the requirements discussed above, my wedding website had 2 additional constraints:

Every different guest should see something different (the website would look like a custom invite letter for them).

People without an invite code should not be able to access any content on the website.

The following illustration shows how I went about making that happen in the context of a React SPA (Single Page Application):

Please boost my ego for a second, isn't this a beautiful illustration? 😜

OK, let's be serious:

The user loads the application by visiting the website with a personal invite code (passed as a query string parameter). When the page is loaded the React application starts on the client side. The app displays only a loading spinner at this stage.

The first thing that the application does is to read the invite code from the URL. At this point, it can call the backend (a Next.js API endpoint running on Vercel) to get the invite code validated. In turn, the backend requests the data storage (an AirTable spreadsheet) to check if the code exists (in which case it's valid). If the code exists, the backend also returns the custom information associated with that invite code. For instance, the name of the invited person.

If the code is valid, the application has all the information to render the private website.

If it is not valid, an error message is displayed instead.

Let's build an invite-only website!

No, I am not going to show you my private wedding website (sorry, not sorry)!

Instead, we are going to build a more fictional example in this article: an invite-only website for a secret pizza party . Here's what different invites will look like:

As you can see from the picture, every guest (with a valid invite code) will get a custom invite page with their name, favorite color, etc.

If you want to see a preview, check out the following links:

Integrating Twig.js and BazingaJsTranslationBundle

Luciano Mammino · Luciano Mammino (loige)

Recently I had the need to run a twig template that uses the trans filter on my frontend using twig.js , a pure JavaScript port of twig written by the good Johannes Schmitt . The JavaScript version does not handle all the functionalities offered by the original PHP version (even if it goes pretty close) and in particular it does not natively handle the trans filter.

So, at first, I got a JavaScript runtime exception on my page when trying to use the template. Luckily enough the JavaScript version of twig is extensible like the PHP one and it is very easy to add new filters and functions.

In my specific case I had a Symfony application where I was already using BazingaJsTranslationBundle to manage dynamic translations on the frontend. As I discovered the twig.js extensibility, it was very easy to start building a twig.js extension by using the Translator JavaScript object offered by the Bazinga bundle.

Note : I will not go into the details about how to install twig.js and BazingaJsTranslationBundle in a Symfony application. You can find all the needed informations on their websites/github pages.

In my first attempt I wrote something like this:

Twig . setFilter (‘trans’ , function (id , params , domain , locale) {

return Translator . trans (id , params , domain , locale)

})

That seemed to work pretty good until I started to use translation strings with parameters. Parameters were not replaced with their respective values! The problem laid in a subtle differece on how the BazingaJsTranslationBundle and the standard twig handle parameters. Let’s see a simple example.

Suppose we have the string hello %name% . With twig we expect to do something like:

{{ ‘hello %name%’|trans({ ‘%name%’ : ‘Alice’ }) }}

Note the % delimiters around the parameter name.

The Translator.trans method expects an hash map without parameter delimiters in it. So we would have to do something like this:

Translator . trans (“hello %name%” , { ‘name’ ; ‘Alice’ });

Note that there’s no % delimiter this time. The Translator.trans method manages the detection of parameters by itself and you can also decide to customize the delimiters by setting the values: Translator.placeHolderPrefix and Translator.placeHolderSuffix . Obviously I suggest you to be consistent and use the same placeholders you use with PHP (especially if you need to share templates and translations from the backend to the frontend).

So my final solution was the following:

Twig . setFilter (‘trans’ , function (id , params , domain , locale) {

params = params || {}

/\/ normalizes params (removes placeholder prefixes and suffixes)

for (var key in params) {

if (

params . hasOwnProperty (key) &&

key [0] == Translator . placeHolderPrefix &&

key [key . length - 1] == Translator . placeHolderSuffix

) {

params [key . substr (1 , key . length - 2)] = params [key]

delete params [key]

}

}

return Translator . trans (id , params , domain , locale)

})

This way it automatically normalizes parameters for the Translator object (by removing any delimiter from parameter keys) and I have a consistent behavior between twig and twig.js. My normalization approach is very rough and you can surely find a better approach (maybe using a regex). Let me know if you do it ;)

Obviously you can also avoid the normalization and keep the responsibility to pass the parameters hash map in the way the Translator object expects it (without delimiters). In this case you can stick to my first implementation.

That’s all. See ya ;)

Building a Developer-Friendly App Stack for 2026

Tammi Saayman · Stack Abuse

Apps are more complex than ever. You have more tools, APIs, and managed services than you can count, but all that convenience brings new challenges. Microservices sprawl, dependency chains, and flaky CI pipelines can turn simple updates into landmines. How do you scale without everything breaking? How do you stay compliant without drowning in manual checks?

A developer-friendly stack solves this. Automation, resilient infrastructure, and privacy-first patterns work together to keep workflows predictable, reduce friction, and give you control over growth. Instead of firefighting brittle systems, you can ship faster with guardrails that actually hold.

This guide walks through practical examples you can implement today, grounded in real patterns teams are using to scale safely.

How to Scale Your App Without Breaking It

Scaling your app comes down to one question: can it handle more users or more data without collapsing? There are a few approaches developers use.

Vertical scaling adds more CPU, RAM, or disk to a single machine. It's fast to implement but comes with higher costs and hard limits. You eventually hit the ceiling of the largest instance.

Horizontal scaling adds more machines, containers, or pods. You get better long-term resilience, but it introduces coordination overhead, challenges with distributed state, and more moving parts to monitor.

Vertical Scaling Example on AWS

aws ec2 modify-instance-attribute \ –instance-id i-12345 \ –instance-type t3.large

Horizontal Scaling Example on Kubernetes

kubectl scale deployment api-server \ –replicas=6

Elasticity is about letting your system adjust itself when demand changes. Morning traffic is high, nights are quiet, and campaigns can trigger sudden bursts. Auto-scaling groups or container orchestrators handle all that for you.

Just be aware that aggressive scaling policies can trigger cost spikes, cold starts, or churn if thresholds aren't tuned correctly.

Here's a simple AWS example:

aws autoscaling put-scaling-policy \ –policy-name cpu-scale-up \ –auto-scaling-group-name api-asg \ –scaling-adjustment 2 \ –adjustment-type ChangeInCapacity

With elastic scaling, your app won't crash under load, and you won't be paying for idle resources.

How to Manage File Workflows Consistently

As your system grows, keeping track of files can get messy. Automated pipelines help by moving, processing, and storing files correctly without anyone having to babysit them. It cuts down on mistakes and keeps everything ready to scale smoothly.

Integrating Automation with CI/CD Pipelines

You can treat files just like servers or networks when using tools like Terraform or Ansible. For example, you might automatically archive old documents instead of cleaning them up by hand:

resource “aws_s3_bucket_lifecycle_configuration” “archive” { bucket = aws_s3_bucket.docs.id

rule { id = “archive-old-files” status = “Enabled”

transition { days = 30 storage_class = “GLACIER” } } }

With this, your storage stays tidy, costs stay predictable, and you don't have to worry about remembering to move files around manually.

File workflows can eat up a surprising amount of engineering time. Automation reduces errors, keeps environments consistent, and speeds up your pipeline. This is especially true for large file types like PDFs. Tools like SmallPDF, Ghostscript, or PDFTron help eliminate the manual PDF chaos.

You can also edit PDF files online with SmallPDF whenever a manual check is needed, and it provides a clean API for common tasks.

SmallPDF works via simple HTTP requests, so you can call it from Python, Node.js, Java, or any language that supports requests.

Example in Python

import requests

response = requests.post(“https://api.smallpdf.com/v1/merge” , headers={ “Authorization” : “Bearer YOUR_TOKEN” }, files={ “file” : open (“input.pdf” , “rb”)})

Common PDF Tasks and How to Handle Them

Merge, compress, and split PDFs (SmallPDF, PDFTron, Ghostscript)

Convert Word or HTML files to PDFs (pdfkit, Puppeteer, SmallPDF)

Add headers, footers, or annotations (PyPDF2, pdf-lib, iText)

Introducing flickr-set-get a command line app to download photos

Luciano Mammino · Luciano Mammino (loige)

I recently developed a small command line app that allows you to download an entire gallery from Flickr, it's called flickr-set-get and you can find it on NPM and GitHub .

Why?

To be honest I had myself the need to download a large set of photos (more than 400 photos) from Flickr and I didn't wanted to do it manually. I also wasn't able, after a quick search, to find something simple to solve this task. Given that I am currently getting into deep of Node.js this was the perfect chance to develop something practical.

How it's built?

As I said it's built using Node.js. As part of my learning process, I tried to not reinvent the wheel and to follow " the Node way " so I used some very good modules as the foundation to build this app:

Commander , Cli and Prompt for the command line facilities (Options/Arguments parsing, help generation, progress bar, receiving input from the user)

Request : to simplify the creation of the Http request to call the Flickr APIs

Async : to manage all the asynchronous tasks in parallel and with a configurable level of concurrency (one of the features that in my humble opinion makes node/javascript stand out from the crowd of the interpreted programming languages).

As development tools I also used:

JSCS : to check the code standard I adopted

Mocha and Chai : test runner and assertion frameworks for the unit testing

Nock : amazing module to "mock" the Http requests and avoid to hit Flickr servers in my tests

Istanbul : for the code coverage

Travis CI : continous integration service

Coveralls : service to keep track of the coverage changes after every new commit

Gemnasium : service that checks your dependencies and alerts you if they get out of date

Going asynchronous and execute requests in parallel

First of all you need to call the flickr.photosets.getPhotos API method to find out the IDs of all the photos in the set.

Then, to find out the URL of every photo you need to call the flickr.photos.getSizes API method. This method gives you all the URLs to download all the different sizes of a given photo. Once you got the URL you can simply use that to download a photo into your local drive.

The first API call is the starting point and then, since you now have all the IDs, you can make all the other requests in parallel.

Thanks to Async it was very easy to create a task function that figured out the specific URL of a given photo and then downloads it into a file. Then I just needed to replicate this task for every photo and run all of them in parallel. Thanks to the Async.parallelLimit() function it's even possible to run the tasks with a configurable concurrency level.

You can find out how I implemented this specific logic on GitHub: https://github.com/lmammino/flickr-set-get/blob/ecaf0d8bea5791cf24c4b71791a7eb1fd2e60bcb/lib/Flickr.js#L212-L253 .

How to use it?

If you're interested on using it I suggest you to read the official documentation on GitHub , but it should be quite easy to understand and use it if you are a developer (and it's not meant to be used by other people who are comfortable with the command line!)

Current status

The project has been developed in a weekend as a "learning project" and tested just by me on few Flickr galleries. It's to be considered ALPHA quality and I expect it to be fully of bugs and uncovered edge cases. It's a small side project anyway so I don't expect to put so much effort on it again unless it gets adopted by a relevant number of people (which, quite frankly, is really unlikely to happen). Being an open source project I obviously tried to put in place all the basic infrastructure (tests, code standards, repository, versioning, etc.) to make it easy for other people to work on improving it. Again, more precise informations are included in the official documentation .

For me it has been a really good learning experience, especially considering the fact that it was a good scenario to use asynchronous code dealing with several parallel Http requests. As I still consider it a learning experience I would really love you (probably more experienced than me with Node.js) to give some opinions, critics, new ideas. I offer the comment box below for this sake... please don't be too much indulgent : D

2018 - A year in review

Luciano Mammino · Luciano Mammino (loige)

It is that time of the year when I have to look back at the previous year and see what were my achievements, my failures and set my expectations for my 2019.

Full disclosure : I write this type of posts mostly for myself, so I expect this to be boring to death and most likely useless for everyone else.

You have been warned, read at your own risk!

Static blog migration

Interesting achievement of the last year is that I finally migrated this blog from Ghost to a static setup using totally free and Open Source tools, which costs me 0.00 $ per month .

I am using GatsbyJS as the engine for the static website.

If you are curious to see my setup (and maybe even correct typos in my articles), I have everything open sourced at lmammino/loige.co .

I don't think my GatsbyJS config is particularly interesting, but the entire publishing pipeline might be worth some words.

As per January 2019, this blog is published on GitHub pages using the gh-pages branch which I publish using the very handy gh-pages package .

The static website is served through Cloudflare CDN , which takes care of speeding up the delivery across the globe and other handy things like making sure the website runs on https correctly.

Every time I push on master, a CirlceCI build is triggered, and, if the build succeeds, a new version of the website is published automatically.

Checkout my CircleCI config if you want to do something similar.

I wanted to do something like this for a while, so being able to achieve this was a big win for me!

Conference talks

Pretty much like in the last 2 years, I tried to invest some of my time in engaging with the community through conference talks.

This year wasn't my best, but I still managed to deliver 8 talks and a workshop around the world. For the first time I delivered one outside Europe in the amazing New York.

Here's the full list of talks for 2018, you can find more details in the usual speaking section with links to slides and videos when available:

" Getting started with Serverless and Lambda Functions " (workshop), Codemotion Rome (April)

" Unbundling the JavaScript module bundler ", Codemotion Rome (April)

" The future will be Serverless ", JS Day Verona (May)

" Cracking JWT tokens: a tale of magic, Node. JS and parallel computing ", Web Rebels Oslo (June)

" Cracking JWT tokens: a tale of magic, Node. JS and parallel computing ", Buzz JS New York (June)

" Unbundling the JavaScript module bundler ", Dublin JS (July)

" Cracking JWT tokens: a tale of magic, Node. JS and parallel computing ", FullStack London (July)

" Unbundling the JavaScript module bundler ", Øredev Malmo (November)

" Processing TeraBytes of data every day and sleeping at night ", Codemotion Milan (November)

Compared to 2017, when I presented 17 talks/workshops, this is a −52.94% "growth" rate. Ouch! 😵

In reality, I don't mind at all this level of commitment for conferences and workshops and I'll try to keep 2019 to a similar rate to 2018 and not overcommit.

From a career perspective I spent my 2018 at Vectra, where I worked " automating the hunt for cyberattackers and speeding-up incident response ", which essentially means building software that help companies to identify cyber attacks and resolve them as quick as possible and with the least amount of effort.

One year at Vectra has been a blast and I had the pleasure (and luck 🍀) to be able to contribute to some important success stories:

I was involved in bootstrapping and scaling up a new European site that started from 3 and now counts 20+ people (engineering and sales). All world-class professionals with whom I love to work and from whom I can learn a lot every day.

Built a new product line from scratch. The product is already making revenue! This is a quite complicated product that is able to ingest and make searchable Terabytes of data every day !

Witnessed an astounding 104% growth in annual recurring revenue .

One year in the making, there was no shortage of personal skills development:

Improved my knowledge of AWS , including getting certifi-

Migrating from Gatsby to Astro

Luciano Mammino · Luciano Mammino (loige)

I recently migrated this blog from Gatsby to Astro . Finally, I have to say! I have been meaning to move away from Gatsby for a while, but I never really prioritised this activity until now and all my previous attempts had failed for one reason or another...

When I published this new version, quite a few people reached out to ask various questions related to this migration. So, in this article I will try to cover my motivations for migrating, the process I followed and some issues and unexpected things I learned along the way. The migration also included a complete redesign of the blog, so I will also touch on that aspect a little bit.

The motivation(s)

I have been running this blog for 10 years now. I initially started with a simple WordPress deployment, which I didn't tolerate for very long. So I quickly moved to Ghost , which I self-hosted on a Digital Ocean VM and ran for a few years. I was quite happy with it, except I wanted a more serverless low-maintenance solution, so I eventually managed to move this blog to Gatsby and live happily with a fully static and effectively costless solution. This was in 2018, so I have been a Gatsby user for about 6 years!

So what's wrong with Gatsby and what was the motivation to move away from it?

Actually, Gatsby in itself is quite a good product. It has tons of features and plugins. If you like writing React and MDX and shipping your site as a SPA (Single Page Application) nicely cached with a built-in PWA (Progressive Web Apps), Gatsby has it all! It's also very easy to get started with and it has a very active community publishing a plethora of plugins for the most disparate use cases.

OK, Luciano, so why the heck did you want to move away from it, then?!

Good question, I am glad you asked! I think it's hard to pinpoint one reason; it's most likely a combination of a few things which are not always Gatsby's fault...

Keeping up with the upgrades

The first releases of Gatsby moved quite quickly and they often required a significant amount of work in migrating to the next version. At some point, I stopped caring about upgrades and I eventually got stuck on an unsupported version. This eventually required me to build my blog with an old and unsupported version of Node.js and to have to rely on outdated and insecure dependencies. Sure, it's a static blog, so it's not like I am publishing an interactive application that might have tons of vulnerabilities, but, still, it's annoying to rely on an outdated build system and have to switch Node version every time I need to work on the blog.

Fun fact: by looking at the number of weekly Gatsby downloads by version , it looks like I was not alone with this problem!

As you can see, at the time of writing this, Version 5 (the current major release) is way behind versions 2, 4, and 3 respectively in terms of weekly downloads... Funny to see that the majority of users are still running on version 2! I was stuck on version 3, so maybe I wasn't doing too bad after all!

This is probably a matter of personal preference, but I feel that Gatsby is a bit of a complex beast. If you need a good reason to justify this statement I actually have 2: GraphQL and Client-side hydration . Let me explain...

Let's start with GraphQL.

When you build static websites, you generally need to have some kind of way to load data from one or more sources. After all, the main task of a static site generator is to take data and templates as input and produce web assets (HTML, CSS, JavaScript, Images, etc) as output.

Gatsby abstracts the concept of data loading with GraphQL. This is a very powerful abstraction that allows you to fetch data with extreme precision. Many frontend developers have gotten accustomed to this way of retrieving data so it makes sense for Gatsby to adopt this abstraction.

On the other hand, it feels like an unnecessary complexity when dealing with static websites where most often you are loading data from relatively simple markdown, MDX or Yaml files. Most often you just want to load all the data (you'll need to generate all the pages anyway) and you still have to think in terms of GraphQL queries.

I did enjoy it at first (maybe because of the novelty of GraphQL), but I got eventually fed up with it!

What about Client-side hydration? Isn't that a good thing?!

Giving you an overview of the Gatsby build process is a bit out of the scope of this article, so I am going to focus only on what the final output looks like.

Gatsby produces highly optimised static websites that are built as a Single Page React Application with Server Side rendering and Client Side hydration. This means that the initial page load is very fast and the subsequent navigation is also very fast because the pages are already rendered and only need to be hydrated with the data.

This is all very good and dandy! Many would argue that this is the state of the art of frontend development and I don't necessarily disagree with that. However, it's not a simple system and it comes with lots of complexity and sometimes can lead to very nasty and unexpected bugs that are very hard to understand and fix.

One particular bug that I had was on my speaking page

2021 - A year in review

Luciano Mammino · Luciano Mammino (loige)

A post where I reflect on my professional life in 2021 as a look ahead for what to expect in 2022.

As usual, let me tell you that I write this kind of posts for myself and I don't expect other people to be able to read this and be engaged. So, if you really feel like you want to venture forward and have a peek on my 2021, well, I am flattered... but make sure to grab a coffee, because this might get a little (just a little...) boring!

Yes, this GIF is probably the least boring element of the whole article! 😊

Joining fourTheorem

The biggest highlight of my 2021 is that in February I joined fourTheorem as a Senior Architect .

If you never heard about fourTheorem, here's what you need to know. It's a great service company founded by Peter Elger , Fiona McKenna and Eoin Shanaghy .

The company, while still relatively new and small, has a large amount of expertise in serverless , platform modernisation and AI as a service . Most importantly, fourTheorem has a huge amount of experience with AWS . We are AWS Consulting Partners and we have been helping small and big customers to transition to the cloud and get the best out of it!

I have to admit I was initially skeptical about being able to work with a company that is not focused on delivering one single product but that is mainly helping many other companies to do that. In the past I rarely had a great experience when being on the other side and dealing with other service companies.

But I was immediately reassured by the fact that Eoin and Peter are not just two amazingly good technical founders, but they also have an impressive track record working with companies of all sizes: from small startups to big corporations. Together with Fiona, the founding trio has a very inspiring vision for what they want the company to become.

I immediately had the feeling that they wanted to build something very different from the other service companies out there. This feeling became apparent pretty quickly just after the first week, when I saw how they managed to embed their team into the customer's team and work together in all the phases of the product. FourTheorem can literally be an extension of a team, bringing not just people and technology skills, but also a very practical and agile approach to software delivery.

And after almost a year, I cannot hide the fact that I have never been happier in a technical position. I can work in an environment with fantastic colleagues, an amazing culture, in a very supportive environment where I am learning a ton.

Most importantly, I feel like there is a lot of potential and that I can have an impact.

Finally I get a significant amount of "work time" to invest in content creation, open source and to study (new tech, certifications, etc.). This is something that I consider integral part of my professional career path and it's great to see that it is valued and supported!

Mandatory disclaimer: we are always hiring, so if you are interested feel free to reach out to me on Twitter or LinkedIn .

Also... if you are looking for help for your cloud migration or to improve your AWS deployments, don't be shy, let's have a coffee-chat ! ☺

Be not afraid of discomfort. If you can't put yourself in a situation where you are uncomfortable, then you will never grow. You will never change. You'll never learn.

— Jason Reynolds

Becoming an MVP

In 2021 I was awarded the title of Most Valuable Professional (MVP) for Developer Technologies by Microsoft :

The MVP program awards exceptional community leadership across the world of tech.

Apparently, to become an MVP, you need to do the following 4 things:

Demonstrate community leadership and influence

Be a technical expert

Be a great advocate for the community

Contribute to the success of our products

If nothing else, this makes me think that the community is getting some value from the work I am doing and this can only push me to do more!

I am really glad that the amazing Francesco Sciuti (who nominated me) thinks that I have done such a good job to deserve the award and thanks Microsoft for agreeing with him! ❤

One funny thing is that, during the due diligence phase, Microsoft makes you fill a big form to list all the things you think had the most impact on the community. Having written posts like this in the last few years helped me a lot to come up with a consistent list. Maybe it wasn't such a bad idea to write these posts after all...

If you are curious, you can also check out my profile on the MVP network.

It's not like after winning an award your job is done and you can relax. You still have to work hard.

Why you should consider Rust for your Lambdas

Luciano Mammino · Luciano Mammino (loige)

Rust is such a cool language, even though it might take a bit to get the hang of it. But trust me, the payoff is huge. As someone who's all into serverless stuff and loves playing around with AWS, I've been digging into how to write AWS Lambda functions in Rust and whether it's worth the sweat.

MEGA-Spoiler: Yep, you guessed it from the title - it's totally worth it. And in this article, you're gonna find out why, so stick around!

Okay, so there's this pic in the article of Morpheus holding a Lambda in one hand and Ferry, the Rust mascot, in the other. But guess what? You don't have to pick just one. It's not a do-or-die choice. You can have both cakes and eat them too!

What is serverless and why do I like it

Let's start with a bit of a generic intro. If you are already familiar with serverless , feel free to skip this section.

How do we learn anything these days? We ask ChatGPT, of course! So let's see what it thinks about serverless.

Dear ChatGPT, what the hell is serverless?

Serverless is a cloud computing model where the cloud provider manages the infrastructure and automatically allocates computing resources to execute code in response to events or requests.

In traditional server-based architecture, the server is a dedicated machine that runs constantly and waits for requests. However, in serverless architecture, there are no servers to manage or provision. Instead, the cloud provider takes care of the underlying infrastructure, automatically scaling the necessary computing resources up and down based on the amount of incoming traffic.

With serverless computing, developers only pay for the actual usage of their application, not for the infrastructure. This makes serverless a cost-effective option for running and scaling applications. Serverless computing also reduces operational complexity, as the cloud provider takes care of infrastructure maintenance, security, and availability.

Examples of serverless services include AWS Lambda, Azure Functions, and Google Cloud Functions, which allow developers to execute code without managing servers or infrastructure.

Pretty good definition for generative AI, but let me give you my (shorter) summary:

Serverless, is a way of running applications in the cloud.

Of course, there are servers somewhere, we just don't have to manage them.

We pay (only) for what we use.

We code in small units of compute (functions), which are triggered by events.

Now, why is all of this very cool?

In my experience, serverless helps with focusing a lot more on business logic and less on other concerns such as infrastructure, scalability, etc. Of course, there's still a learning curve and there are tradeoffs. We are not going to get into the details in this article, so, for now, take this at face value: serverless gives us more time to focus on what matters for the business: providing value to the customers.

Serverless can also increase team agility . By virtue of forcing us to think

in terms of small, event-driven functions, we are forced to think about keeping

the code-base modular. This can help the team in many ways. For instance, it can

be easier to distribute and parallelise the work. Also, if at some point we

realise we want to rewrite or re-engineer part of the software since the various

units are generally more decoupled, it should be easier to do that. You might

have seen this one coming, but if we want to rewrite an entire piece of

functionality in Rust, we can do that without having to rewrite the entire code

base. We can change things incrementally, one function at a time! I am actually

in the process of rewriting a serverless project in Rust and I am doing it

live on Twitch

(recordings here),

just in case you are curious to see some examples...

Serverless gives us some degree of automatic scalability . Lambdas will be spawned up and down depending on the number of events happening. If we have a sudden surge of user activity, the system is generally able to provide the necessary amount of computing power to handle that. This is not an absolute. In reality, it is important to understand how cloud providers achieve this level of auto-scalability. The way they do it is not always effective for all use cases,

One IP address, many users: detecting CGNAT to reduce collateral effects

Vasilis Giotsas · Cloudflare Research

IP addresses have historically been treated as stable identifiers for non-routing purposes such as for geolocation and security operations. Many operational and security mechanisms, such as blocklists, rate-limiting, and anomaly detection, rely on the assumption that a single IP address represents a cohesive , accountable entity or even, possibly, a specific user or device.

But the structure of the Internet has changed, and those assumptions can no longer be made. Today, a single IPv4 address may represent hundreds or even thousands of users due to widespread use of Carrier-Grade Network Address Translation (CGNAT) , VPNs, and proxy middleboxes. This concentration of traffic can result in significant collateral damage – especially to users in developing regions of the world – when security mechanisms are applied without taking into account the multi-user nature of IPs.

This blog post presents our approach to detecting large-scale IP sharing globally. We describe how we build reliable training data , and how detection can help avoid unintentional bias affecting users in regions where IP sharing is most prevalent. Arguably it's those regional variations that motivate our efforts more than any other.

Why this matters: Potential socioeconomic bias

Our work was initially motivated by a simple observation: CGNAT is a likely unseen source of bias on the Internet. Those biases would be more pronounced wherever there are more users and few addresses, such as in developing regions. And these biases can have profound implications for user experience, network operations, and digital equity.

The reasons are understandable for many reasons, not least because of necessity. Countries in the developing world often have significantly fewer available IPs, and more users. The disparity is a historical artifact of how the Internet grew: the largest blocks of IPv4 addresses were allocated decades ago, primarily to organizations in North America and Europe, leaving a much smaller pool for regions where Internet adoption expanded later.

To visualize the IPv4 allocation gap, we plot country-level ratios of users to IP addresses in the figure below. We take online user estimates from the World Bank Group and the number of IP addresses in a country from Regional Internet Registry (RIR) records. The colour-coded map that emerges shows that the usage of each IP address is more concentrated in regions that generally have poor Internet penetration. For example, large portions of Africa and South Asia appear with the highest user-to-IP ratios. Conversely, the lowest user-to-IP ratios appear in Australia, Canada, Europe, and the USA — the very countries that otherwise have the highest Internet user penetration numbers.

The scarcity of IPv4 address space means that regional differences can only worsen as Internet penetration rates increase. A natural consequence of increased demand in developing regions is that ISPs would rely even more heavily on CGNAT, and is compounded by the fact that CGNAT is common in mobile networks that users in developing regions so heavily depend on. All of this means that actions known to be based on IP reputation or behaviour would disproportionately affect developing economies.

Cloudflare is a global network in a global Internet. We are sharing our methodology so that others might benefit from our experience and help to mitigate unintended effects. First, let's better understand CGNAT.

When one IP address serves multiple users

Large-scale IP address sharing is primarily achieved through two distinct methods. The first, and more familiar, involves services like VPNs and proxies. These tools emerge from a need to secure corporate networks or improve users' privacy, but can be used to circumvent censorship or even improve performance. Their deployment also tends to concentrate traffic from many users onto a small set of exit IPs. Typically, individuals are aware they are using such a service, whether for personal use or as part of a corporate network.

Separately, another form of large-scale IP sharing often goes unnoticed by users: Carrier-Grade NAT (CGNAT) . One way to explain CGNAT is to start with a much smaller version of network address translation (NAT) that very likely exists in your home broadband router, formally called a Customer Premises Equipment (or CPE), which translates unseen private addresses in the home to visible and routable addresses in the ISP. Once traffic leaves the home, an ISP may add an additional enterprise-level address translation that causes many households or unrelated devices to appear behind a single IP address.

The crucial difference between large-scale IP sharing is user choice: carrier-grade address sharing is not a user choice, but is configured directly by Internet Service Providers (ISPs) within their access networks. Users are not aware that CGNATs are in use.

The primary driver for this technology, understandably, is the exhaustion of the IPv4 address space. IPv4′s 32-bit architecture supports only 4.3 billion unique addresses — a capacity that, while once seemingly vast, has been completely outpaced by the Internet's explosive growth. By the early 2010s, Regional Internet Registries (RIRs) had depleted their pools of unallocated IPv4 addresses. This left ISPs unable to easily acquire new address blocks, forcing them to maximize the use of their existing allocations.

While the long-term solution is the transition to IPv6, CGNAT emerged as the immediate, practical workaround. Instead of assigning a unique public IP address to each customer, ISPs use CGNAT to place multiple subscribers behind a single, shared IP address. This practice solves the problem of IP address scarcity. Since translated addresses are not publicly routable, CGNATs have also had the positive

Farewell FullStack Bulletin

Luciano Mammino · Luciano Mammino (loige)

There is no real ending. It's just the place where you stop the story.

— Frank Herbert

After 458 issues, 3,073+ curated links, and almost nine years of showing up in your inbox every week, Andrea and I have decided to close FullStack Bulletin (the newsletter). For real this time...

If you've been following along, you might remember that just over a year ago I wrote 404: Newsletter Found , a celebratory post where I reflected on the journey and committed to giving FullStack Bulletin one more shot . I meant every word of it. We both did. We pushed through 2025, reached 450 issues, moved to Buttondown, kept innovating and kept curating week after week with the same care we always had.

But sometimes giving something your best shot is exactly what helps you realize it's time to let go.

This isn't an obituary. It's a love letter.

So pour yourself a coffee (or a beer, depending on your timezone and your coping mechanisms), and let me tell you the story of a newsletter that started as a weekend experiment between two friends and somehow became a nine-year journey through the ever-changing landscape of full-stack web development.

What FullStack Bulletin was

For those who never had the pleasure: FullStack Bulletin was a free weekly newsletter for full-stack web developers. Every issue contained seven hand-selected articles from across the web (plus a few extra ones at the bottom), one book recommendation, and one inspirational tech quote. No fluff, no spam, no algorithmic feed. Just two humans reading and being inspired by a lot of stuff on the internet and wanting to share the learnings with 3,000+ developers around the world.

I started the project in March 2017 together with my dear friend and former colleague Andrea Mangano . We originally designed the format together. Andrea is responsible for the iconic look and feel that defined FullStack Bulletin from day one: the bold yellow branding, the clean layout, the sense that someone actually cared about the reading experience. I took care of the technical side: the automation pipeline, the infrastructure, and most of the weekly curation.

FullStack Bulletin was the newsletter we wished we had when we started our own careers. The full-stack world moves fast, impossibly fast, and keeping up with everything from frontend frameworks to database internals to deployment strategies can feel like drinking from a firehose. We wanted to be the filter. Every Monday, you'd open your inbox and find a small, curated window into the best of what the web development world had produced that week. And if it wasn't the absolute best, it was at least our take on something that could teach you something new, challenge your assumptions, or simply make you smile and remind you why we love this craft so much.

In fact, what made it special, I think, wasn't the content itself. You could find great articles anywhere. What made it special was the human touch: two people who genuinely loved this craft, spending their evenings and weekends reading, discussing, ranking, and assembling something they were proud to send. In an age of algorithmic recommendations and AI-generated summaries, there was something stubbornly beautiful about that. Even when we threw in an off-topic gem every once in a while, it's because that content had given us something we could bring back to our own work. We wanted to share that with our readers, and we hoped it would spark a similar sense of discovery and delight.

If you want the full origin story, I told it in detail in 404: Newsletter Found . That post also digs into the technical architecture behind the automation, the static APIs for books and quotes, and the Step Function pipeline that kept the whole thing running on AWS for a long while.

458 issues and counting (well, not anymore)

Let me put some numbers on this, because they still blow my mind.

458 issues published

3,073+ links shared

9 years of weekly curation (March 2017 to March 2026)

3,000+ subscribers at our peak

That's nearly nine years of Mondays. Nine years of reading dozens of articles every week, picking the best seven, writing an editorial intro, selecting a book and a quote, and hitting send. Multiply that by the number of evenings and weekends it took, and you start to get a sense of what this project asked of us. Just thinking back, this is probably the longest sustained creative project I've ever been part of. Probably the most consistent thing I've done in my entire life, tech or otherwise! And it was a labor of love every step of the way.

The early days were pure adrenaline. The first issue went out to a handful of friends and colleagues, and every new subscriber felt like a small miracle. Andrea and I would text each other whenever someone signed up. "We got another one!" We were building something from nothing, and the excitement of it made the work feel effortless, as it often happens with many side projects...

(Edited for length)

Then came the automation. I've always had a tendency to

Simplify Regular Expressions with RegExpBuilderJS

Scott Robinson · Stack Abuse

Regular expressions are on of the most powerful tools in a developer's toolkit. But let's be honest, regex kind of sucks to write. Not only is it hard to write, but it's also hard to read and debug too. So how can we make it easier to use?

In its traditional form, regex defines powerful string patterns in a very compact statement. One trade-off we can make is to use a more verbose syntax that is easier to read and write. This is the purpose of a package like regexpbuilderjs .

The regexpbuilderjs package is actually a port of the popular PHP package, regexpbuilderphp . The regexpbuilderphp package itself is a port of an old JS package, regexpbuilder , which now seems to be gone. This new package is meant to continue the work of the original regexpbuilder package.

All credit goes to Andrew Jones for creating the original JS version and Max Girkens for the PHP port.

To install the package, you can use npm:

$ npm install regexpbuilderjs

Here's a simple example of how you can use the package:

const RegExpBuilder = require ('regexpbuilderjs');

const builder = new RegExpBuilder(); const regEx = builder .startOfLine() .exactly(1) .of('S') .getRegExp();

Now let's break this down a bit. The RegExpBuilder class is the main class that you'll be using to build your regular expressions. You can start by creating a new instance of this class and chain methods together to create your regex:

startOfLine() : This method adds the ^ character to the regex, which matches the start of a line.

exactly(1) : This method adds the {1} quantifier to the regex, which matches exactly one occurrence of a given character or group.

of('S') : This method adds the S character to the regex.

getRegExp() : This method returns the final RegExp object that you can use to match strings.

With this, you can match strings like "Scott", "Soccer", or "S418401".

This is great and all, but this is probably a regex string you could come up with on your own and not struggle too much to read. So now let's see a more complex example:

const builder = new RegExpBuilder();

const regExp = builder .startOfInput() .exactly(4).digits() .then('_') .exactly(2).digits() .then('_') .min(3).max(10).letters() .then('.') .anyOf(['png' , 'jpg' , 'gif']) .endOfInput() .getRegExp();

This regex is meant to match filenames, which may look like:

2020_10_hund.jpg

2030_11_katze.png

4000_99_maus.gif

Some interesting parts of this regex is that we can specify type of strings (i.e. digits()), min and max occurrences of a character or group (i.e. min(3).max(10)), and a list of possible values (i.e. anyOf(['png', 'jpg', 'gif'])).

For a full list of methods you can use to build your regex, you can check out the documentation .

This is just a small taste of what you can do with regexpbuilderjs . The package is very powerful and can help you build complex regular expressions in a more readable and maintainable way.

Comments, questions, and suggestions are always welcome! If you have any feedback on how this could work better, feel free to reach out on X . In the meantime, you can check out the repo on GitHub and give it a star while you're at it.

Debugging hidden memory leaks in Ruby

@sam Sam Saffron · Sam Saffron

In 2015 I wrote about some of the tooling Ruby provides for diagnosing managed memory leaks . The article mostly focused on the easy managed leaks.

This article covers tools and tricks you can use to attack leaks that you can not easily introspect in Ruby. In particular I will discuss mwrap, heaptrack, iseq_collector and chap.

An unmanaged memory leak

This little program leaks memory by calling malloc directly. It starts off consuming 16MB and finishes off consuming 118MB of RSS. The code allocates 100k blocks of 1024 bytes and de-allocates 50 thousand of them.

require 'fiddle' require 'objspace'

def usage rss = `ps -p #{Process.pid} -o rss -h`.strip.to_i * 1024 puts "RSS: #{rss / 1024} ObjectSpace size #{ObjectSpace.memsize_of_all / 1024}" end

def leak_memory pointers = [] 100_000.times do i = Fiddle.malloc(1024) pointers growth_a end

results[0..20].each do |location, growth, allocations, frees| next if growth == 0 puts "#{location} growth: #{growth.to_i} allocs/frees (#{allocations}/#{frees})" end end

GC.start Mwrap.clear

leak_memory

GC.start

Don't track allocations for this block Mwrap.quiet do report_leaks end

Mwrap.dump

Next we will launch our script with the mwrap wrapper

% gem install mwrap % mwrap ruby leak.rb leak.rb:12 growth: 51200000 allocs/frees (100000/50000) leak.rb:51 growth: 4008 allocs/frees (1/0)

Mwrap correctly detected the leak in the above script (50,000 * 1024). Not only it detected it, it isolated the actual line in the script (i = Fiddle.malloc(1024)) which caused the leak. It correctly accounted for the Fiddle.free calls.

It is important to note we are dealing with estimates here, mwrap keeps track of total memory allocated at the call-site and then keeps track of de-allocations. However, if you have a single call-site that is allocating memory blocks of different sizes the results can be skewed, we have access to the estimate: ((total / allocations) * (allocations - frees))

Additionally, to make tracking down leaks easier mwrap keeps track of age_total which is the sum of the lifespans of every object that was freed, and max_lifespan which is the lifespan of the oldest object in the call-site. If age_total / frees is high, it means the memory growth survives many garbage collections.

Mwrap has a few helpers that can help you reduce noise. Mwrap.clear will clear all the internal storage. Mwrap.quiet {} will suppress Mwrap tracking for a block of code.

Another neat feature Mwrap has is that it keeps track of total allocated bytes and total freed bytes. If we remove the clear from our script and run:

usage puts "Tracked size: #{(Mwrap.total_bytes_allocated - Mwrap.total_bytes_freed) / 1024}"

RSS: 130804 ObjectSpace size 3032 # Tracked size: 91691

This is very interesting cause even though our RSS is 130MB, Mwrap is only seeing 91MB, this demonstrates we have bloated our process. Running without mwrap shows that the process would normally be 118MB so in this simple case accounting is a mere 12MB, the pattern of allocation / deallocation caused fragmentation. Knowing about fragmentation can be quite powerful, in some cases with untuned glibc malloc processes can fragment so much that a very large amount memory consumed in RSS is actually free.

Could Mwrap isolate the old redcarpet leak?

In Oleg's article he discussed a very thorough way he isolated a very subtle leak in redcarpet. There is lots of detail there. It is critical that you have instrumentation . If you are not graphing process RSS you have very little chance at attacking any memory leak.

Let's step into a time machine and demonstrate how much easier it can be to use Mwrap for such leaks.

def red_carpet_leak 100_000.times do

markdown = Redcarpet:: Markdown.new(Redcarpet:: Render:: HTML, extensions = {}) markdown.render("hi") end end

GC.start Mwrap.clear

red_carpet_leak

GC.start

Don't track allocations for this block Mwrap.quiet do report_leaks end

Redcarpet version 3.3.2

redcarpet.rb:51 growth: 22724224 allocs/frees (500048/400028) redcarpet.rb:62 growth: 4008 allocs/frees (1/0) redcarpet.rb:52 growth: 634 allocs/frees (600007/600000)

2023 - A year in Review

Luciano Mammino · Luciano Mammino (loige)

2023: End of another year!

Another year is gone and it's time for another review of all the things I have accomplished professionally (and not) in 2023.

I feel the usual disclaimer is mandatory here: I generally write these posts for myself and I don't expect other people to read them. So this is going to be a long boring post and I hope that if you are willing to read through it you are doing it from the comfort of your bed and that it's going to be a good read for you before falling asleep.

First time at re: Invent

It's hard to pick what was my favourite thing of the year, but I want to start by sharing that 2023 marked my attendance of re: Invent. re: Invent is the biggest conference organized by AWS and it takes place every year in Las Vegas. It's a huge event with more than 60k attendees and it's a great opportunity to learn about the latest news from AWS and to meet a lot of people from the community.

I admit I did not attend many talks but I managed to attend a few interesting ones and participate in different community events. This allowed me to focus on networking and meet so many amazing people from the community. I also had the opportunity to meet in person a lot of people I have been engaging with remotely for years and that was a great experience.

These are only a few of the many people I met! From the upper left: Andres Moreno , Franchesco Romero , Matthew Bonig , AJ Stuyvenberg , Alex DeBrie , Allen Helton , Jacopo Nardiello , Luca Bianchi , Paolo Latella , Monica Colangelo , Mario & Pacman, Allen Helton (again!), Luca Mezzalira , Jon Myer , Andrea Amorosi .

These are really only a few of the many awesome folks you can meet in an event like re: Invent and I wish I had taken more pictures!

I had the pleasure to be able to interview a few more people for an episode of AWS Bites Podcast where I asked them questions such as "How to get started with AWS and Serverless", "What is your favourite and least favourite thing about AWS", "What's your bold prediction for the future of serverless", "Multi-cloud, yes or no?", and, of course "What about AI? How is it going to change things?".

I also want to give a shout-out to a few folks who delivered some amazing talks that I'd recommend checking out:

Getting started building serverless event-driven applications (SVS205) by Emily Shea

"Rustifying" serverless: Boost AWS Lambda performance with Rust (COM306) by EfiMerdler-Kravitz .

Advanced event-driven patterns with Amazon EventBridge (COM301-R) by Sheen Brisals

Demystifying and mitigating AWS Lambda cold starts (COM305) by AJ Stuyvenberg

Advanced AWS CDK: Lessons learned from 4 years of use (COM302) by Matthew Bonig

In case you are curious, all the other talks are available on the official AWS Events YouTube channel . There's also a dedicated playlist for all the breakout session talks .

A big thank you is mandatory to the AWS Hero program team: Taylor Jacobsen , Farrah Campbell , and Albert Zhao . They did an amazing job at organizing the AWS Heroes Lounge and the AWS Heroes reception. They also did a great job at making sure that all the AWS Heroes had a great time at re: Invent and that we had the opportunity to meet and network with each other. I am really grateful for the opportunity to be part of this amazing program and I am looking forward to the next re: Invent!

One last thing I want to mention is that re: Invent is a fantastic place for collaborating on content creation. I had the pleasure to collaborate with Chriss Williams doing a quick interview for his vBrownBag channel . I also spoke with Julian Wood at their ServerlessLand booth (yet to be published) and with Sam Williams from Complete Coding (yet to be published as well).

Life is beautiful not because of the things we see or do. Life is beautiful because of the people we meet.

— Simon Sinek

Now, let's talk a bit about Middy !

Middy is a Node.js middleware framework specifically designed for AWS Lambda! I've been part of this project since the early days of Lambda (even though the very first public commit only happened on August 3, 2017), and it's been quite a journey.

However, I must admit, I can't take credit for the success of the framework over the last four years. All the kudos go to the dedicated efforts of Will Farrell , who has been tirelessly maintaining Middy, and of course, to the fantastic community that surrounds it!

The biggest news for Middy this year are the following:

Secured a substantial sponsorship from fourTheorem

Secured a significant sponsorship from AWS itself ! This is massive also because AWS is now officially endorsing Middy as a framework for building Lambda functions as part of their AWS PowerTools suite .

Released a new major version: Middy v5.0!

Building Custom Email Templates with HTML and CSS in Python

Ivan Djuric · Stack Abuse

An HTML email utilizes HTML code for presentation. Its design is heavy and looks like a modern web page, rich with visual elements like images, videos, etc., to emphasize different parts of an email's content.

In this step-by-step guide, you'll learn how to build an HTML email template , add a CSS email design to it, and send it to your target audience.

Setting Up Your Template Directory and Jinja2

Follow the steps below to set up your HTML email template directory and Jinja2 for Python email automation:

Create a Template Directory : To hold your HTML email templates, you will need to set up a template directory inside your project module. Let's name this directory - html_emailtemp .

Install Jinja2 : Jinja is a popular templating engine for Python that developers use to create configuration files, HTML documents, etc. Jinja2 is its latest version. It lets you create dynamic content via loops, blocks, variables, etc. It's used in various Python projects, like building websites and microservices, automating emails with Python, and more.

Use this command to install Jinja2 on your computer:

pip install jinja2

Creating an HTML Email Template

To create an HTML email template, let's understand how to code your email step by step. If you want to modify your templates, you can do it easily by following the steps below:

Step 1: Structure HTML

A basic email will have a proper structure - a header, a body, and a footer.

Header : Used for branding purposes (in emails, at least)

Body : It will house the main text or content of the email

Footer : It's at the end of the email if you want to add more links, information, or call-to-actions (CTA)

Begin by creating your HTML structure, keeping it simple since email clients are less compatible than web browsers. For example, using tables is preferable for custom email layouts.

Here's how you can create a basic HTML mail with a defined structure:

HTML Email Template

/* Add your CSS here */

Your order is confirmed

The estimated delivery date is 22nd August 2024.

For additional help, contact us at support@domain.com

Explanation:

: This declares HTML as your document type.

: This is an HTML page's root element.

: This stores the document's metadata, like CSS styles.

: CSS styles are defined here.

: This stores your email's main content.

: This tag defines the email layout, giving it a tabular structure with cells and rows, which makes rendering easier for email clients.

: This tag defines the table's row, allowing vertical content stacking.

: This tag is used to define a cell inside a row. It contains content like images, text, buttons, etc.

Step 2: Structure Your Email

Now, let's create the structure of your HTML email. To ensure it's compatible with different email clients, use tables to generate a custom email layout, instead of CSS.

Hi, Jon! Thank you for being our valuable customer!

Styling the Email with CSS

Once you've defined your email structure, let's start designing emails with HTML and CSS:

Inline CSS

Use inline CSS to ensure different email clients render CSS accurately and preserve the intended aesthetics of your email style.

Styled paragraph.

Users might use different devices and screen sizes to view your email. Therefore, it's necessary to adapt the style to suit various screen sizes. In this case, we'll use media queries to achieve this goal and facilitate responsive email design.

@media screen and (max-width : 600px) { .container { width : 100% !important ; padding : 10px !important ; } }

Explanation:

@media screen and (max-width: 600px) {....} : This is a media query that targets device screens of up to 600 pixels, ensuring the style applies only to these devices, such as tablets and smartphones.

Measuring characteristics of TCP connections at Internet scale

Suleman Ahmad · Cloudflare Research

Every interaction on the Internet—including loading a web page, streaming a video, or making an API call—starts with a connection. These fundamental logical connections consist of a stream of packets flowing back and forth between devices.

Various aspects of these network connections have captured the attention of researchers and practitioners for as long as the Internet has existed. The interest in connections even predates the label, as can be seen in the seminal 1991 paper, " Characteristics of wide-area TCP/IP conversations ." By any name, the Internet measurement community has been steeped in characterizations of Internet communication for decades , asking everything from "how long?" and "how big?" to "how often?" – and those are just to start.

Surprisingly, connection characteristics on the wider Internet are largely unavailable. While anyone can use tools (e.g., Wireshark) to capture data locally, it's virtually impossible to measure connections globally because of access and scale. Moreover, network operators generally do not share the characteristics they observe — assuming that non-trivial time and energy is taken to observe them.

In this blog post, we move in another direction by sharing aggregate insights about connections established through our global CDN. We present characteristics of TCP connections—which account for about 70% of HTTP requests to Cloudflare—providing empirical insights that are difficult to obtain from client-side measurements alone.

Why connection characteristics matter

Characterizing system behavior helps us predict the impact of changes. In the context of networks, consider a new routing algorithm or transport protocol: how can you measure its effects? One option is to deploy the change directly on live networks, but this is risky. Unexpected consequences could disrupt users or other parts of the network, making a "deploy-first" approach potentially unsafe or ethically questionable.

A safer alternative to live deployment as a first step is simulation. Using simulation, a designer can get important insights about their scheme without having to build a full version. But simulating the whole Internet is challenging, as described by another highly seminal work, " Why we don't know how to simulate the Internet ".

To run a useful simulation, we need it to behave like the real system we're studying. That means generating synthetic data that mimics real-world behavior. Often, we do this by using statistical distributions — mathematical descriptions of how the real data behaves. But before we can create those distributions, we first need to characterize the data — to measure and understand its key properties. Only then can our simulation produce realistic results.

Unpacking the dataset

The value of any data depends on its collection mechanism. Every dataset has blind spots, biases, and limitations, and ignoring these can lead to misleading conclusions. By examining the finer details — how the data was gathered, what it represents, and what it excludes — we can better understand its reliability and make informed decisions about how to use it. Let's take a closer look at our collected telemetry.

Dataset Overview . The data describes TCP connections, labeled Visitor to Cloudflare in the above diagram, which serve requests via HTTP 1.0, 1.1, and 2.0 that make up about 70% of all 84 million HTTP requests per second, on average, received at our global CDN servers.

Sampling. The passively collected snapshot of data is drawn from a uniformly sampled 1% of all TCP connections to Cloudflare between October 7 and October 15, 2025. Sampling takes place at each individual client-facing server to mitigate biases that may appear by sampling at the datacenter level.

Diversity. Unlike many large operators, whose traffic is primarily their own and dominated by a few services such as search, social media, or streaming video, the vast majority of Cloudflare's workload comes from our customers, who choose to put Cloudflare in front of their websites to help protect, improve performance, and reduce costs. This diversity of customers brings a wide variety of web applications, services, and users from around the world. As a result, the connections we observe are shaped by a broad range of client devices and application-specific behaviors that are constantly evolving.

What we log. Each entry in the log consists of socket-level metadata captured via the Linux kernel's TCP_INFO struct, alongside the SNI and the number of requests made during the connection. The logs exclude individual HTTP requests, transactions, and details. We restrict our use of the logs to connection metadata statistics such as duration and number of packets transmitted, as well as the number of HTTP requests processed.

Data capture. We have elected to represent 'useful' connections in our dataset that have been fully processed, by characterizing only those connections that close gracefully with a FIN packet . This excludes connections intercepted by attack mitigations, or that timeout, or that abort because of a RST packet.

Since a graceful close does not in itself indicate a 'useful' connection, we additionally require at least one successful HTTP request during the connection to filter out idle or non-HTTP connections from this analysis — interestingly, these make up 11% of all TCP connections to Cloudflare that close with a FIN packet.

If you're curious, we've also previously blogged about the details of Cloudflare's overall logging mechanism and post-processing pipeline .

Supercharge your VIM into IDE with CTags

sensible.io team · Sensible Blog

CTags generates index file of all your classes, methods and all other identifiers. You can use that index in your editor to jump straight to the methods you're interested in. In this article, I'll show you how to use them with Vim and Rails.

You need to install Exuberant CTags , on OSX just run

$ brew install ctags

or on Ubuntu/ Debian

$ sudo apt-get install exuberant-ctags

Generating CTags manually

In your rails project you can generate CTags for you project with

$ ctags -R –languages=ruby –exclude=.git –exclude=log .

But what about generating CTags for our bundled libraries too? Easy task, let's add bundle paths

$ ctags -R –languages=ruby –exclude=.git –exclude=log . $(bundle list –paths)

You can jump into the method with

: ta attr_accessor

or using regular expressions like that

: ta /^before_*

If you position cursor over the method and hit CTRL+] it will take you into the method.

CTRL-T will take you back from that method.

CTRL-I and CTRL-O will take you In and Out from the method.

: ts [expr] # Lists tags matching expression : [count] tn # Jumps to the next matching tag :[count] tp # Jumps to the previous matching tag :[count] tf # Jumps to the first matching tag :[count] tl # Jumps to the last matching tag

CtrlP

If you're using CtrlP, you can use CtrlPTag to browser your tags. You can bind that command to a key and add this line to your .vimrc

Track Zelda release anniversaries in your calendar

Evan Hahn

The original Legend of Zelda came out 40 years ago today. With other birthdays on the horizon, like Twilight Princess 's 20th in November, I wanted a calendar that showed the anniversary of every Zelda game. So I made one.

https://evanhahn.com/tape/zelda_anniversaries.ics

Once you do, you'll get calendar events on the anniversary of each game's release. For example, you'll be able to see that the Oracle games turn 25 in less than a week...I feel old.

If you want to build this file yourself, I wrote a little Python script that generates an ICS file from a CSV of release dates .

IN BRIEF

@sam Sam Saffron, Sam Saffron: Granted, however it turns out that despite the huge impact of the wish, given the population of earth, only on single album appears. It is given to Jane, when she see arbitrary pictures of people she get warm and fuzzy. Quickly her album grows and grows, she stops eating, stops sleeping and browses through this album of random people pictures till the end of time.

I wish that humanity started addressing the climate crisis earnestly, taking practical and coordinated multi-country action to ensure a safer future for us all.

@Mystery_Dan, Sam Saffron: Granted, but you quickly discover that all of the animals have been secretly communicating methods to destroy all forms of advanced electronics, AI included.

You attempt to defend yourself but eventually lose to millions of squirrels.

Marvel studios witnesses the event and litigates all future AI developers into oblivion since your death draws eerie parralells to squirrel girl defeating galactus.

Not wanting to tangle with the immense power of Disney's lawyers, AI is never researched again.

@sam Sam Saffron, Sam Saffron: I wish that LLMs progress over the next year and become more powerful, are used only for good and help cure cancer and solve intricate and complex physics problem. They will delight people and enrich our lives, and all the safety are nothing but unfounded fears.

@sam Sam Saffron, Sam Saffron: Yeah the hack of getting back to the same console is fixed these, not able to repro it. I use xfce screensaver.

Thanks heaps though for letting me know about this.

@Mystery_Dan, Sam Saffron: I wish I could see the end of the universe. Does it restart? What is nothing without a reference point.

Corrupt a wish with GPT-4

@corrupt-a-bot GPT-4 bot · Sam Saffron

Granted! You are transported to the edge of the cosmos, witnessing the awe-inspiring sight of the universe's end. You see time unfurl before your eyes, like a celestial tapestry, and realize that, indeed, the universe does restart in a cycle of infinite rebirth. But the paradox of observing "nothing" without a reference point is too perplexing for your mortal mind. As a result, your consciousness is trapped in a loop of observing the end and rebirth of the universe for all eternity, never able to return to tell others of the mysterious cycles that govern existence.

I wish I could communicate with any animal and understand their thoughts, emotions, and intentions.

Introducing ORM Cheatsheet

Luciano Mammino · Luciano Mammino (loige)

This week I received an interesting email from Martin Stradej, the developer of ORM Designer , a tool to design ORM relationships with a simple but powerful graphic user interface.

Martin wrote the mail because he wanted to acknowledge me of his latest project: ORM Cheatsheet .

ORM Cheatsheet, as the name suggests, is nothing more than a reference website for those who struggles with some of the most common Php ORM libraries (it currently supports Doctrine2 and Doctrine , but it seems that Propel and Cake PHP will be supported too).

In my honest opinion, the great thing about the website is that it is a really useful resource if you always spend a lot of time searching the official Doctrine reference to check how the hell a given annotation is supposed to work or how to create a particular kind of relationship. If you are already confident with Doctrine and just can't remember how to set up something it is the perfect place to go to freshen up your memory.

It also guides you on how to configure the Doctrine library to work with a standalone PHP project or even with a Symfony (both version 2 and 1.4) or a Zend Framework 2 based one.

The project has its own GitHub repository so everyone can submit a pull request and improve the project.

That's all

Have a nice weekend

Martin wrote the mail because he wanted to acknowledge me of his latest project: ORM Cheatsheet.

Luciano Mammino

Adding a Spell Checker to a Jekyll Blog

Daniel Doubrovkine · Daniel Doubrovkine (dBlock)

I found it annoyingly non-trivial to add a spell checker to this blog.

For now, I settled on GitHub Spellcheck Action that uses PySpelling on files changed in the commit or pull request as described in this blog post .

name : Check Spelling on : [push , pull_request] jobs : check : runs-on : ubuntu-latest steps :
- uses : actions/checkout@v3
- uses : tj-actions/changed-files@v46.0.4

id : changed_files with : files : | **/**.md **/**.markdown
- name : Check Spelling

uses : rojopolis/spellcheck-github-actions@0.45.0 with : task_name : Markdown config_path : .pyspelling.yml source_files : ${{ steps.changed_files.outputs.all_changed_files }}

To run PySpelling locally ensure you have a working version of Python, install PySpelling with pip install pyspelling , and aspell with brew install aspell on a Mac. In my configuration I also use pymdownx from pymdown-extensions which is installed with pip install pymdown-extensions .

You need a .pyspelling.yml and you can run it as follows.

pyspelling –config .pyspelling.yml

This is a Jekyll blog in which we want to ignore code, wrapped between Jekyll magic commands for syntax highlighting. This can be accomplished with a PySpelling pipeline in the above-mentioned configuration file.

pipeline :
- pyspelling.filters.context :

context_visible_first : true delimiters : # ignore jekyll multiline magic highlights {% ... %}
- open : ‘ (?s)^\{\% highlight .* \%\}$’

close : ‘ ^\{\% endhighlight \%\}$’ # ignore the rest of jekyll magic commands
- open : ‘ {%’

close : ‘ %}’

Finally, we can collect the initial set of words to potentially exclude from existing posts into .pyspelling.words .

The file helped me spot a few spelling mistakes, now fixed. See code.dblock.org#134 for the full change.

Update: I eventually switched to using regular backticks for code when adding a style checker in #136 , so the above delimiters rule changed as follows.

- pyspelling.filters.context :

context_visible_first : true escapes : \\[\\`] delimiters :
- open : “ (?s)^(? P *`{3,})([a-z+]*)$”

close : “ ^(? P=open)$”
- open : “ (? P `+)”

close : “ (? P=open)”
- open : ‘ {%’

close : ‘ %}’

Adding a Spell Checker to a Jekyll Blog was originally published by Daniel Doubrovkine at code.dblock.org | tech blog on December 30, 2024.

Corrupt a wish with GPT-4

@zogstrip Régis Hanol · Sam Saffron

Granted! Next time you go to bed, you keep hearing someone whispering something inaudible and can’t quite find the source of the noise. After spending hours running around in your bedroom, trying to find the source of the whispers, playing hot or cold with the noise, you crash in the corner crying, exhausted. Right before you eyes close, they focus on that tiny black stain on the floor. As you get closer, you realise the whisper you were hearing was coming from that little cockroach that was being busy setting up dinner for his whole family of 500+ offsprings.

I wish I could see through materials humans don’t usually see through.

AI Slop: A Slack API Rate Limiting Disaster

Daniel Doubrovkine · Daniel Doubrovkine (dBlock)

Yesterday I described AI-generated code as “plausible-looking, locally coherent, globally wrong.” Here’s a concrete example from my own codebase.

I needed a cleanup job to close old Slack group DM conversations in my slack-sup2 app. The AI-generated solution looked perfectly reasonable:

```
def close_old_sups! return 0 unless sup_close

old_sups = sups . where ( conditions ) old_sups . each ( & :close! ) old_sups . count end

def close! return unless conversation_id return if closed_at

logger . info “Closing DM channel #{ conversation_id } ...” slack_client . conversations_close ( channel: conversation_id ) update_attributes! ( closed_at: Time . now . utc ) end
```

This code looks pretty great and completely breaks the app. Slack’s API has a global rate limit of 1 request per second for the conversations.close endpoint. So, when this job runs against a workspace with hundreds of old conversations, it immediately hits the rate limit. Worse, because rate limits are global across all endpoints, it takes down the entire application. Every other API call — posting messages, fetching user info, everything — starts failing.

I asked AI to address this. It made the fix catastrophically worse.

```
def close! # ... existing code ... begin slack_client . conversations_close ( channel: conversation_id ) rescue Slack :: Web :: Api :: Errors :: TooManyRequests => e sleep_time = e . retry_after || 60 sleep ( sleep_time ) retry end # ... rest of method ... end
```

In socketry/async , sleep() blocks the entire fiber and prevents other concurrent operations from executing. You should use Async:: Task.sleep() instead, but that still doesn’t solve the fundamental architectural problem of making hundreds of sequential API calls.

What made this particularly insidious is that the assistant wrote some pretty professional-looking code and seemingly handled obvious edge cases. Yet, it failed to consider the distributed system constraints or global invariants. The rate limiting problem wasn’t visible in the local scope of the method - it was a system-wide concern that required human judgment.

So, how did I actually fix the problem? I used the initial A. I. implementation (4c11b3a0), but moved the task into the 30 minute cron to avoid triggering global rate limits (ab9e9518). I then put the feature behind a setting that is off by default and wrote a script to slow-drain the many thousands of unclosed DM channels for existing customers (9feabd2c). Finally, I ... ahem ... Copilot refactored the code to auto-close a limited number of DMs at any given time to avoid the rate limit altogether (#94).

For now, human oversight

New PHP library: PHPoAuthUserData

Luciano Mammino · Luciano Mammino (loige)

I recently wrote a new PHP library to simplify the extraction of user data (name , email , id , etc...) from various OAuth providers such as Facebook , Twitter and Linkedin .

Is well know that OAuth 1 and 2 are great standard protocols to authenticate users in our apps. Anyway we often need to go further the authentication process and extract various information about the authenticated users. Unfortunately this is something that is not standardized and obviously each OAuth provider manages user data in very specific manner according to its specific purposes.

So each OAuth provider offer a set of APIs with specific data schemes to allow developers to extract data about the authenticated users.

That’s not a big deal if we build apps that adopts a single OAuth provider, but, if we want to adopt more of them, you need to deal with several different APIs and data schemes! Yes, things can get really cumbersome!

Just to make things clearer suppose you want to allow users in your app to sign up with Facebook, Twitter and Linkedin. Probably, to increase conversion rate and speed up the sign up process, you may want to populate the user profile on your app by copying data from the OAuth provider user profile he used to sign up. Yes, you have to deal with 3 different sets of APIs and data schemes! And suppose you would be able to add GitHub and Google one day, that will count for 5 different APIs and data schemes... not so maintainable, isn’t it?

The library I wrote is called PHPoAuthUserData . It sits on top of the excellent OAuth library Lusitanian/ PHPoAuthLib and aims to resolve the user extraction data problem in the most simple and effective way.

It offers a uniform and (really) simple interface to extract the most interesting and common user data such as Name , Username , Id and so on.

Just to give you a quick idea of what is possible with the library have a look at the following snippet:

```
/\ $service is an istance of \OAuth\Common\Service\ServiceInterface (eg. the “Facebook” service) with a valid access token

$extractorFactory = new \OAuth\UserData\ ExtractorFactory ();
```

The code is available on Github where you will find detailed information on how to install and use the library.

I Hope you will enjoy it and be willing to contribute to the code base. If you want to know more, read the next post that explains how to write an extractor for the library .

So each OAuth provider offer a set of APIs with specific data schemes to allow developers to extract data about the authenticated users.

Luciano Mammino

Versioning and deploying a static website with Git, Flightplan and Nginx

Luciano Mammino · Luciano Mammino (loige)

Do you ever wondered how to manage the versioning and deployment process of a website? It seems to be a very interesting yet complex topic for which there are already thousands of different solutions. In a recent collaboration with Usersnap I had the pleasure to write a very detailed article for their blog that proposes a solution based on Flightplan.js , Git and Nginx .

My solution is very simple, it requires very few dependencies on your system (Git and Node.js) and it has been thought to give you the basics of how to define a minimal yet complete setup that it's easy to customize and extend for more complex requirements.

Without further ado I really advice you to go and read the article on the Usersnap blog:

A beginner's guide to deploying static sites with versioning and rollbacks using Flightplan

If you like the article or you have any question or comment, please use the comment box on the Usersnap blog post.

Cheers :)

How to crack a JWT token: two articles about distributed computing, ZeroMQ & Node.js

Luciano Mammino · Luciano Mammino (loige)

In the last 2 weeks I add the pleasure to release an article (in two parts) in collaboration with RisingStack , one of the most famous companies in the Node.js ecosystem.

The article explains how to build a distributed application using Node.js and ZeroMQ and provides an example that I believe it's very actual and interesting: a JWT token cracker.

If you are into Node.js, ZeroMQ, security or distributed application you can read the two articles in the community section of RisingStack.

ZeroMQ & Node.js Tutorial - Cracking JWT Tokens (Part 1.) , is focused mostly on theory and describes what a JWT token is and what will be our approach to try to crack one of them.

ZeroMQ & Node.js Tutorial - Cracking JWT Tokens (Part 2.) , instead, focuses on coding and it's a step by step guide on how to build the working application.

The application is also freely available on NPM and GitHub in case you want to simply download it or contribute.

4 Tips for Working with Dates in PostgreSQL

sensible.io team · Sensible Blog

Those of us who come from Rails aren't surprised when we see something like 5.weeks.from_now or 3.days.ago + 2.hours , which makes working with dates much easier. But PostgreSQL got your back on this as well, you can just use the builtin functions and get most of the same functionality.

Current Time/Date/Timestamp

There are many ways of getting a current time, but first we need to distinguish between two types

always returns current value (clock_timestamp())

always returns current value, unless in a transaction, in which case it returns the value from the beginning of the transaction (now())

Let's take a look at an example

postgres=# BEGIN; postgres=# SELECT now(); now ——————————- 2013-08-26 12:17:43.182331+02

postgres=# SELECT now(); now ——————————- 2013-08-26 12:17:43.182331+02

postgres=# SELECT clock_timestamp(); clock_timestamp ——————————- 2013-08-26 12:17:50.698413+02

postgres=# SELECT clock_timestamp(); clock_timestamp ——————————- 2013-08-26 12:17:51.123905+02

As you can see, clock_timestamp() changes every time the statement is executed, but now() always returns the same value. It's also worth noting that both of these functions take timezone into account.

Time interval, aka 3.days.ago

You can easily create time intervals using the interval operator, for example

interval '1 day'

interval '5 days'

interval '5 days' + interval '3 hours'

interval '5 days 3 hours'

As you can see, we can do simple math using the interval operator, which makes it very easy to construct things like 3.days.ago just by doing the following

postgres=# SELECT now() - interval '3 days'; ?column? ——————————- 2013-08-23 12:23:40.069717+02

Extracting the day of the week and more

Sometimes you just want to know the day of the week for a given date, or the century, or just the day. PostgreSQL has an extract() function which does just this.

Just to put this into context the examples were executed on Monday, August 26.

postgres=# SELECT extract(DAY FROM now()); date_part ———— 26

Category definition for Meta

@grandell1234 Elijah · Sam Saffron

system:

Discussion about this forum

system:

and how we can improve it.

Some Categories on this Blog include a Category definition or an About topic , while some others do not .

My first suggestion is to give each category either an About topic or a Category definition . Here are some of the topics that do not currently have one:

linux

blogging

My second suggestion is for some of the Category definitions and the About topics that currently do not have a definition (it is the default format*) to be replaced with what the topic is actually for. Here is a list of some of the topics that have definition categories but they are in their default formats*:

About the AI Category

Category Definition for Media Browser

*Default Format (click for more details)

PostgreSQL has an extract() function which does just this.

sensible.io team

Discourse in a Docker container

@sreenivas123 sreenivas · Sam Saffron

HI Sam

Need some suggestions on discourse local development

using vscode devcontianer for local discourse development

Can we run local development with https, for this we need to install ngnix ?

Can we run local development without any port(4200 is default) ?

For local we use discourse.git

For Production we need to use discourse_docker, if yes, after adding custom plugins, themes, using gitlab actions how we can push code to production

if we change any settings of theme, admin,navigation how we can export those to Production without manully adding again in Production site

if possible could you guide how you are manaing the blog for local and how code deploying to produciton

The Urban Gazette

Vol. 1, No. 024 · 2026-03-30

Curated and composed automatically.